SUPER AI ENGINEER HACKATHON

The Ad Ecosystem and CTR prediction

From brand budget to browser render
in under 100 milliseconds

100ms
per auction

$600B
global ad spend

~15%
avg CTR

Who Does What in the Ad Ecosystem

Advertiser

BRAND / BUSINESS

Has a message and a budget. Wants to reach the right person at the right moment.

DSP

DEMAND-SIDE PLATFORM

Buys inventory on the advertiser's behalf. Runs the CTR model and submits bids.

Ad Exchange

RTB MARKETPLACE

Neutral auction venue where SSPs and DSPs transact in real time.

SSP

SUPPLY-SIDE PLATFORM

Sells publisher inventory programmatically. Maximises the price each slot fetches.

Publisher

WEBSITE / APP

Owns the audience and the ad slots. Earns revenue when inventory is sold.

DMP / CDP

DATA PLATFORM

Supplies audience segments and behavioural signals to both sides of the market.

One Impression. Six Steps. 100 Milliseconds.

1

0 ms

User Loads Page

User opens an app or website. The publisher's SSP detects an available ad slot.

`publisher_id · site_id / app_id`

2

5 ms

Bid Request Broadcast

The SSP sends the impression opportunity to multiple ad exchanges with all available context signals.

`banner_pos · device_type · hour`

3

20 ms

DSPs Run CTR Models

Each DSP runs its own prediction model. Predicted CTR × click value = bid price.

`historical_user_ctr · C1 / C15 · ad_campaign_id`

4

80 ms

Second-Price Auction

Highest bidder wins but pays the second-highest price + 1¢. This incentivises honest bidding.

`ad_id · publisher_id`

5

95 ms

Ad Rendered

Winning creative is returned and displayed before the page finishes loading.

`creative_size · impression_id`

6

∞

Click Signal Logged

If the user clicks, a positive training example is recorded — closing the feedback loop.

`clicked = 1`

Files

- train.csv : Weeks 1–3 of impression data with ground-truth labels
- test.csv : Week 4 impression data — predict `clicked` for each row
- sample_submission.csv

Features (1)

IDENTIFIERS

Column	Type	Description
impression_id	string	Unique identifier for each impression
user_id	string	Anonymised user hash — no external meaning

TEMPORAL

Column	Type	Description
timestamp	datetime	Date and time the impression was served
hour	categorical	Hour of day (0–23)
day_of_week	categorical	Day of week (Mon–Sun)

DEVICE

Column	Type	Description
device_type	categorical	Mobile, Desktop, or Tablet
device_model	categorical	Device model (e.g. iPhone14)
device_conn_type	categorical	WiFi, 4G, 3G, 2G, or Ethernet

IMPRESSION CONTEXT

Column	Type	Description
banner_pos	categorical	Ad banner position on page: 0, 1, 2, or 7
is_app	boolean	True = app impression; False = site

Features (2)

SITE FEATURES

Populated only when is_app = False. Null for app impressions.

Column	Type	Description
`site_id`	string	Anonymised website identifier
`site_domain`	categorical	Coarser domain grouping of site_id
`site_category`	categorical	News, Sports, Gaming, Finance…

APP FEATURES

Populated only when is_app = True. Null for site impressions.

Column	Type	Description
`app_id`	string	Anonymised mobile app identifier
`app_domain`	categorical	Coarser domain grouping of app_id

AD FEATURES

Column	Type	Description
`ad_id`	string	Anonymised ad creative identifier
`ad_campaign_id`	string	Campaign identifier (~2 000 unique)
`publisher_id`	string	Publisher identifier (~1 000 unique)
`creative_size`	categorical	300x250, 728x90, 160x600, 320x50, 300x600
`ad_quality_score`	float	Composite score computed from DMP and CDP data.

ANONYMISED FEATURES

Column	Type	Description
`C1`	categorical	7 unique values — treat as opaque signal
`C15`	categorical	5 unique values — treat as opaque signal
`C16`	categorical	6 unique values — treat as opaque signal
`C21`	categorical	8 unique values — treat as opaque signal

Features (3)

ENGAGEMENT

Column	Type	Description
`user_depth`	integer	Pages visited in session before this impression (1–30)
`user_segment`	string	Pages Categorical feature derived from user browsing behavior.
`historical_user_ctr`	float	User's historical CTR — averaged across past impressions

TARGET VARIABLE

Column	Type	Description
`clicked`	boolean / float	1 = clicked; submit as a probability (0–1)

Your Dataset Columns, Mapped to Real-World Concepts

`impression_id`	→ One RTB auction event — a single bid request	`ad_id`	→ The specific creative submitted to the auction
`user_id`	→ Anonymised token passed in the bid request	`banner_pos`	→ Slot position — above-fold slots command higher prices
`publisher_id`	→ The SSP / publisher selling the ad slot	`device_conn_type`	→ Bid request signal — 4G users behave differently from WiFi
`site_id / app_id`	→ Specific property in the publisher's portfolio	`historical_user_ctr`	→ Feature the DSP engineers from its own click logs
`ad_campaign_id`	→ Advertiser's campaign running through a DSP	`clicked`	→ Ground truth observed after the auction settles

Notes

- Null values are not errors
- Check decoy features
- Be careful about frequency
- Be careful of leaky features
- Test set is in the Future!

Evaluation Metric

- Normalized Cross-Entropy (NCE)

$$\mathrm{NCE} = \frac{\mathrm{LogLoss}(y, \hat{y})}{\mathrm{LogLoss}(y, \bar{p})}$$

  - y is the ground-truth
  - $\hat{y}$ is the prediction
  - $\bar{p}$ is training set click-through rate (CTR), naïve baseline

- NCE = 1 ⟶ model is no better than naïve baseline
- NCE < 1 ⟶ you add some predictive value
- NCE = 0 ⟶ perfect

- **Submission format: your predicted `clicked` must be a probability between 0 and 1, not a binary label.**

Evaluation Metric

```
from sklearn.metrics import log_loss

def normalized_cross_entropy(y_true: np.ndarray,
                             y_pred: np.ndarray,
                             base_ctr: float | None = None) -> float:
    """
    NCE = model_logloss / baseline_logloss
    Baseline predicts the training set CTR for every impression.
    Lower is better; < 1.0 means the model beats the naive baseline.
    """
    if base_ctr is None:
        base_ctr = float(np.mean(y_true))
    baseline_pred  = np.full_like(y_pred, base_ctr)
    model_logloss  = log_loss(y_true, y_pred)
    baseline_logloss = log_loss(y_true, baseline_pred)
    return model_logloss / baseline_logloss
```

Rules

- Individual
- 6 submissions
- Until 23:55

Have fun